Common Task Framework Rules

Section 1: Research Methodology Rules

Rule 1: Temporal Integrity (No Lookahead Bias)

All information used for portfolio construction at time t must be available at or before time t. Portfolio weights cannot use future information in any form.

Using future information to make predictions is known as **lookahead bias** and invalidates results. This includes:

- Using future returns, prices, or characteristics in feature construction
- Selecting hyperparameters based on test period performance
- Choosing features based on their known historical performance
- Any form of data leakage from the prediction target

> **Example:**
>
> You cannot use returns from June 2025 to create a portfolio in May 2025. All information used for portfolio construction at time t must be available at or before time t.

> **Automated Testing:**

We employ automated tests to detect lookahead bias. These tests compare model outputs when trained on different subsets of data. A properly constructed model will produce identical predictions for any given time period regardless of what future data was available during training.

How the test works: Your model is run on the full dataset, then again on a truncated dataset (with future data removed). If the weights differ beyond floating-point tolerance, lookahead bias is detected and the model will be flagged for review.

Rule 2: Feature Selection Constraints

Feature selection must be entirely algorithmic. Manual selection of features based on historical performance knowledge is prohibited. All feature engineering must be performed programmatically within the submitted code.

Prohibited Example:

Creating a model that exclusively uses 12-month return momentum and book-to-market equity to build a portfolio. Such selections could introduce lookahead bias if the feature choices were influenced by their known historical performance.

Permitted Example:

Building a portfolio based on the ten best-performing characteristics at time t, where the selection is determined algorithmically using only information available at that point in time.

Rule 3: Data Sources

Only the provided CTF dataset is permitted. External data sources, including macroeconomic indicators, alternative data, or web-scraped information, are prohibited. Feature engineering through mathematical transformations of provided characteristics is permitted.

> **Prohibited Example:**
>
> Using external macroeconomic data, alternative datasets, or web-scraped information to enhance your model.
>
> **Permitted Example:**
>
> Creating new features by transforming or combining existing characteristics in the provided dataset (e.g., ratios, moving averages, or interaction terms).

Rule 4: Reproducibility Requirements

Submissions must be fully reproducible. All code must be self-contained with complete dependency specifications.

- **Python**: Use `requirements.txt` or `pyproject.toml` for package dependencies
- **R**: Use `renv.lock` for reproducible package versions

Dependencies must be available from PyPI (Python) or CRAN (R). Private package repositories or local packages are not supported. See Rule 8 for pre-installed packages.

> **Best Practices:**
>
> - Pin exact package versions (e.g., `statsmodels==0.14.0` not `statsmodels>=0.14`)

- Consider using `uv` (Python) or `renv` (R) for precise dependency management and reproducibility
- Test your submission in a fresh environment before uploading

Rule 5: Technical Implementation

Required submission components: Model Script (required), Portfolio Weights (generated), and Methodology Document (highly encouraged).

- **Model Script** (required): A self-contained Python or R script that implements the `main()` function with the specified signature (see Rules 11-12)
- **Portfolio Weights** (generated): CSV file with portfolio weights for all observations where `ctff_test` is True
- **Methodology Document** (highly encouraged): A PDF describing your approach.

Rule 6: Portfolio Construction

Portfolios must be rebalanced monthly. No constraints are imposed on portfolio construction methodology. Shorting, leverage, position limits, and turnover constraints are permitted at the submitter's discretion.

Rule 7: Academic Integrity

Submissions of prior published work are encouraged. Multiple submissions are permitted to allow iterative improvement. Standard academic citation practices are recommended for methodology descriptions.

Section 2: Execution Environment Rules

Rule 8: Pre-Installed Packages

Each execution environment includes a baseline set of packages. Additional dependencies may be specified in your dependency file.

Runtime environment: submissions run on **Python 3.13** (`.py` models) or **R 4.4.2** (`.r` models). Any pinned dependency must be compatible with this runtime. A pinned Python package must provide a wheel for Python 3.13. A version that requires a newer Python (for example, one published only for 3.14 and above) has no installable build and will fail the build step.

Python Pre-Installed Packages (each at the latest version compatible with Python 3.13):

- `pandas`
- `numpy`
- `pyarrow`
- `boto3`
- `scipy`
- `scikit-learn`
- `polars`
- `joblib`

R Pre-Installed Packages (each at the latest version compatible with R 4.4.2):

- `arrow` (required for Parquet I/O)
- `data.table` (high-performance data manipulation)
- `dplyr` (tidyverse data manipulation)
- `tidyr` (data tidying)

Additional packages may be installed by including a `requirements.txt` (Python) or `renv.lock` (R) file with your submission. All additional packages must pass security scanning before installation.

Rule 9: Compute Resources and Time Limits

Submissions execute on high-performance computing infrastructure with the following resource allocations:

Resource	Specification
CPU Cores	32
Memory	300 GB RAM
Execution Time Limit	24 hours
GPU	1 GPU available upon request (optional)

Submissions that exceed memory limits will terminate with an out-of-memory error. Submissions that exceed the time limit will be terminated and marked as failed.

GPU Access: If your model requires GPU acceleration, check the "Request GPU Access" option on the submission form. Please also mention your GPU requirements in your model documentation (PDF) to help us optimize resource allocation.

Best Practices:

- Use vectorized operations over explicit loops
- Consider memory-efficient data types (e.g., `float32` instead of `float64`)
- Test locally with a validation dataset before full submission

⚠ Parallelization Warning

If using `joblib` for parallel processing, **avoid the `threading` backend**. Use

the default `loky` backend instead:

```
Parallel(n_jobs=-1, backend='loky')(...)  # Recommended
Parallel(n_jobs=-1, backend='threading')(...)  # May hang
```

The threading backend may cause deadlocks when combined with NumPy/pandas/scikit-learn due to nested thread conflicts.

Note: Resource specifications may be updated. Check this page for current values.

Rule 10: Network Isolation

Submitted code executes in a fully isolated network environment:

- **No outbound network access**: Attempts to connect to external URLs, APIs, or services will fail
- **No internet connectivity**: The execution environment has no route to the public internet
- **Build-time network access**: Network access is available only during the container build phase for package installation

Any code that requires runtime network access will fail. All data required for model execution is provided via the function arguments.

Section 3: Submission Format Rules

Rule 11: Main Function Signature

Your submission must define a `main` function with the exact signature specified for your language:

Python R

```python
def main(chars: pd.DataFrame, features: pd.DataFrame, daily_ret: pd.DataFra
    """
    Args:
        chars: Stock characteristics (ctff_chars.parquet)
        features: Computed features (ctff_features.parquet)
        daily_ret: Historical daily returns (ctff_daily_ret.parquet)

    Returns:
        DataFrame with columns: id, eom, w
    """
    # Your model logic
    return output_df
```

Submissions without a valid `main` function will fail validation.

Rule 12: Output Format Requirements

Your `main()` function must return a DataFrame with the following schema:

Column	Type	Description
`id`	integer	Security identifier from the input data
`eom`	date	End of month date (YYYY-MM-DD format)
`w`	float	Portfolio weight

> **About the `id` column:**
>
> The `id` values come from the input data and must be returned unchanged. For CRSP securities, the `id` is the CRSP `permno`. For Compustat securities, the `id` is a composite identifier. Your output should

> use the same `id` values present in the input DataFrames.

Example output:

id	eom	w
10006	2024-01-31	0.05
17566	2024-01-31	0.03
38914	2024-01-31	-0.02

Validation Requirements:

- DataFrame must be non-empty
- Column names must be exactly `id`, `eom`, `w` (case-sensitive)
- No missing values in any column
- Output size must not exceed 50 MB when serialized

Date Handling:

- Only rows where `ctff_test` is True in the input data affect your score
- Your model may output weights for all dates, but only test period dates are used for scoring

The pipeline infrastructure automatically captures your function's return value and writes it to the output file. Your results must be returned exclusively via the `main()` function's return value. Do not attempt startup scripts, container entrypoints, or direct output file writing.

Rule 13: Source File Constraints

Source code files must meet the following requirements:

Constraint	Limit
Maximum file size	1 MB per file
File encoding	UTF-8
Binary files	Not permitted (source files only)

Files exceeding these limits will be rejected during validation.

Rule 14: Data Input

Your `main()` function receives three DataFrames as arguments, loaded from the following Parquet files:

Argument	Source File	Description
`chars`	`ctff_chars.parquet`	Stock characteristics (fundamental data)
`features`	`ctff_features.parquet`	Computed features (technical indicators)
`daily_ret`	`ctff_daily_ret.parquet`	Historical daily returns

The pipeline loads these files and passes them to your function. You do not need to read files directly.

> **Execution Modes:**
>
> - **Validation**: Uses a small subset (~4 MB) for quick testing
> - **Full**: Uses the complete dataset (~1.1 GB) for final scoring

⚠ **Important:** The validation dataset contains **123 monthly observations**. Your code must execute its main model logic and produce valid output with this smaller dataset, just as it would with the full dataset. If your algorithm requires a minimum number of observations (e.g., for model training or rolling windows), ensure it can handle at least 123 months. The portfolio weights produced during validation are **not used for scoring**. The validation run only verifies that your code executes correctly and produces output in the expected format.

The `CTF_EXECUTION_MODE` environment variable indicates which mode is running, but most models do not need to check this.

Section 4: Security Rules

Rule 15: Prohibited Operations

The following operations are prohibited and will cause submission rejection:

Network Operations:

- `socket`, `urllib`, `requests`, `http.client` (Python)
- `download.file()`, `url()`, `httr` calls (R)

Shell Execution:

- `subprocess`, `os.system`, `os.popen` (Python)
- `system()`, `system2()`, `shell()` (R)

Dynamic Code Execution:

- `eval()`, `exec()`, `compile()` (Python)
- `eval()`, `parse()` with arbitrary strings (R)

Filesystem Access:

- Reading or writing files outside permitted temporary directories
- Attempts to access system files, environment secrets, or other submissions

Credential Exposure:

- Hardcoded API keys, passwords, or tokens in source code

> **Warning:**
>
> Submissions containing these patterns will fail security validation.

Rule 16: Dependency Security Scanning

All package dependencies are scanned for known vulnerabilities before installation:

Scanning Tools:

- **Python**: OSV-Scanner, pip-audit, GuardDog (malware detection)
- **R**: ROSV (R OSV wrapper)

> **Important:**
>
> Submissions with dependencies containing HIGH or CRITICAL severity vulnerabilities will be rejected. If you believe your submission was incorrectly rejected, contact the administrators.

Section 5: Execution Process Rules

Rule 17: Logging and Debugging

Standard output and error streams from your code are captured:

- Use `print()` (Python) or `print()`/`cat()` (R) for debugging output
- Adding logging output helps CTF administrators diagnose issues with your submission
- Avoid logging sensitive information

Consider logging progress updates, timing information, and intermediate results to aid troubleshooting.

Rule 18: Determinism and Reproducibility

Submissions must be fully deterministic. Given the same input data, your model must produce identical outputs every time it is executed.

Requirements:

- **No uncontrolled randomness**: If your model uses random number generation, you must set explicit seeds (e.g., `np.random.seed(42)` or `set.seed(42)`) at the start of your `main()` function
- **Deterministic operations**: Avoid operations with non-deterministic ordering (e.g., iterating over unordered sets or dictionaries without sorting)
- **Reproducible within tolerance**: Outputs must match within floating-point tolerance (relative tolerance: 1e-5, absolute tolerance: 1e-8)

> **Why This Matters:**
>
> Determinism is essential for validating model integrity. We run automated tests that compare model outputs under different conditions. Models that produce different results on each run cannot be properly validated and may be flagged for review.

> **Best Practices:**
>
> - Set all random seeds at the very start of your `main()` function
> - Run your model multiple times locally to verify consistent output

- Use sorted iterations when processing collections
- Pin exact versions of all dependencies to prevent behavior changes

Rule 19: Administrative Code Adjustments

CTF administrators may make minor modifications to submitted code, such as:

- Adding debugging statements
- Correcting execution issues
- Ensuring rule compliance

Submitters will be notified of any substantive changes.

Section 6: Author Identity

Rule 20: Author Identification

We encourage submitters to use their real first and last names.

Submissions where the author cannot be identified may be held back from the leaderboard.

Section 7: Public Disclosure

Rule 21: Public Leaderboard Display

By submitting to the CTF competition, you agree that your qualifying submission will be made publicly available on the leaderboard.

This includes:

- **Your name** (as entered in the submission form)
- **Your model name**
- **Your code file** (.py or .r) — downloadable by anyone
- **Your documentation** (.pdf, if provided) — downloadable by anyone
- **Performance metrics** (Sharpe ratio, returns, volatility, etc.)

This openness allows researchers to learn from successful approaches and recognizes your contribution.

> **What Not to Submit:**
>
> Do not submit code or documentation containing confidential or proprietary information, such as trade secrets, information subject to NDA, or personal data.

Rule 22: Withdrawal Requests

You may request removal of your submission from the leaderboard at any time.

To request withdrawal, submit a request through our contact form using the email address associated with your submission. We will send a confirmation email to verify your identity before processing the request. Upon verification, we will:

- Remove your entry from the public leaderboard
- Remove downloadable files from our servers

> **Important:**
>
> Removal does not affect copies that have already been downloaded by

> third parties or cached by search engines.

Section 8: Model Integrity & Review Process

Rule 23: Review Process

Models may be flagged for review if they exhibit unusual characteristics or fail automated integrity tests.

Reasons for flagging:

- Failed automated lookahead bias tests
- Unusual performance patterns
- Suspected data snooping
- Code review findings

What happens when a model is flagged:

- The model is moved to the "Under Review" section of the leaderboard
- It remains visible but is not included in the main rankings
- The submitter is notified (if notification was enabled)
- The submitter may revise and resubmit their model

> **Appeals:** If you believe your model was flagged in error, please contact us. We are happy to discuss the specific concerns and work with you to resolve them.

Example Implementation

The following examples provide starter templates. Click to expand.

Complete Model Implementation

ECDF transformation and OLS regression example

Python R

```python
import numpy as np
import pandas as pd


def ecdf_transform(data: pd.Series) -> pd.Series:
    """Transform data to empirical CDF values."""
    if data.empty:
        return data
    return data.rank(method='min', pct=True)


def prepare_data(chars: pd.DataFrame, features: pd.Series, eom: str) -> pd.Data
    """Apply ECDF transformation grouped by end-of-month."""
    for feature in features:
        is_zero = chars[feature] == 0
        chars[feature] = chars.groupby(eom)[feature].transform(ecdf_transform)
        chars.loc[is_zero, feature] = 0
        chars[feature] = chars[feature].fillna(0.5)
    return chars


def fit_ols(train: pd.DataFrame, features: pd.Series) -> np.ndarray:
    """Fit an OLS regression on the training data."""
    X = train[features].values
    X = np.column_stack([np.ones(X.shape[0]), X])
    y = train['ret_exc_lead1m'].values
    coeffs, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
    return coeffs


def main(chars: pd.DataFrame, features: pd.DataFrame, daily_ret: pd.DataFrame)
    """Main function to prepare data, fit OLS, and calculate portfolio weights.
    eom = 'eom'
    features = features['features']
```

```
    chars = prepare_data(chars, features, eom)

    train = chars[chars['ctff_test'] == False]
    test = chars[chars['ctff_test'] == True].copy()

    coeffs = fit_ols(train, features)

    X_test = test[features].values
    X_test = np.column_stack([np.ones(X_test.shape[0]), X_test])
    test['pred'] = X_test @ coeffs

    test['rank'] = test.groupby(eom)['pred'].rank(ascending=False, method='aver
    test['rank'] = test.groupby(eom)['rank'].transform(lambda x: x - x.mean())
    test['w'] = test.groupby(eom)['rank'].transform(lambda x: x / x.abs().sum()

    return test[['id', eom, 'w']]
```

Local Testing Setup

How to run your model locally before submission

Important:

This code is **NOT executed** by the CTF pipeline. It is ignored during evaluation. Place only local testing code here. Your model logic must be in the `main()` function.

Python R

```
# Add this at the bottom of your script for local testing
if __name__ == "__main__":
    chars = pd.read_parquet('ctff_chars.parquet')
    features = pd.read_parquet('ctff_features.parquet')
    daily_ret = pd.read_parquet('ctff_daily_ret.parquet')

    pf = main(chars, features, daily_ret)
    pf.to_csv('output.csv', index=False)
```

Note: These rules are designed to ensure the academic integrity, security, and real-world applicability of submitted models. Adherence to these guidelines is essential for meaningful comparative analysis within the Common Task Framework. Rules and resource specifications may be updated; please check this page regularly for the most current requirements.

© 2026 Jensen, T., Kelly, B., and Pedersen, L. Analysis code is available under the MIT License. Data is licensed under CC BY-NC 4.0.

Discussions Repository

Contact

Join our community of researchers